

G05 / PROTECTED ANSWERS

G05 REVIEW A

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first 3

Architecture answer 3

Kafka/key answer 3

Consumer answer 3

Replay answer 3

Delta answer 4

Spark answer 4

Lag answer 4

Practical answer 4

G05 REVIEW A - IMPLEMENTATION RUBRIC 5

Protected runtime rubric A 5

Read this first

Open this book after saving your own attempt. Compare the reasoning and result, repair the cause of a mistake, then close the answer and try changed inputs.

Protected review. Read only after a saved gate attempt. Repair the weak concept, close this file, then take the fresh changed gate.

Architecture answer

Kafka is the durable event-log/transport boundary; Spark Structured Streaming is compute; Parquet is a columnar file format; Delta adds transactional table semantics; Bronze/Silver/Gold are quality/serving contracts. A correct design does not call all of these a database. Cold-chain sensor events can first enter Kafka, Spark can parse/enrich them, Bronze can preserve raw evidence, Silver can enforce clean device/event contracts, and Gold can serve operational aggregates.

Validation: draw the data flow and label where replay evidence lives. If Kafka retention expires, Bronze/raw storage should still support longer repair if that is part of the SLA. The answer fails if it cannot say which layer owns transport, computation, transactional table state and business-serving grain.

Kafka/key answer

Use device_id when per-device temperature order matters. That keeps a device history within one partition under the normal stable-key routing assumption. It does not create global order across all devices, and a single extremely noisy device can create a hot partition.

Validation: state the exact required order (per device), show topic/partition/offset evidence for one device, and check partition-level event distribution. If the key produces severe skew, redesign the business boundary rather than pretending extra consumers can split one key across partitions.

Consumer answer

For an SMS alert side effect, Kafka offset or transaction semantics alone do not make the external provider exactly-once. Create durable alert intent/work state keyed by event_id/alert_id, or use a provider-supported idempotency key. Advance Kafka progress only after a durable handled outcome exists.

Crash-window proof: simulate remote success followed by process failure before offset commit. On replay the idempotency state/provider key must return the already-applied outcome instead of sending a second SMS. That is the evidence that matters, not merely a green consumer process.

Replay answer

Identify the exact source partitions/offsets or time window, current Spark checkpoint, target Delta version and known bad transformation. Prefer an isolated corrected replay or an idempotent MERGE into a controlled target. Do not delete production checkpoints or rewind a live group blindly.

Before promotion reconcile raw/Kafka input evidence to valid, quarantine and corrected output counts/keys. Preserve the old table version and replay manifest so rollback remains possible. A replay is incomplete until you can prove which rows/events changed and which did not.

Delta answer

MERGE with a stable business/event key can make replay idempotent at the Delta table boundary. Table history/version evidence helps compare before/after state and reason about rollback. This does not solve an external side effect automatically.

Validation: replay the same corrected input twice in a safe training target and prove the second run does not create a second business row/effect. Compare key counts and table history. If duplicates grow, the claimed idempotency boundary is false.

Spark answer

A Structured Streaming checkpoint owns query recovery progress and, for stateful operations, state metadata. Deleting it changes the query recovery identity and can cause reprocessing or lost state assumptions. `startingOffsets` is not a universal override for an established checkpoint.

Validation: record checkpoint path, source offsets and output evidence before restart. Restart with the same checkpoint in a disposable lab and observe resume behavior. Any production recovery must preserve or intentionally replace checkpoint identity with a documented replay plan.

Lag answer

If only one partition lags, prioritize key skew/hot key, poison or unusually expensive records, partition-specific broker/network conditions, and sink behavior tied to that data. Adding consumers cannot make two group members simultaneously own the same lagging partition in the normal model.

Validation: trend log-end and committed offsets per partition, inspect key distribution and processing/sink latency, apply one safe correction, then prove lag falls while business reconciliation remains correct. Catching up by dropping bad events is not recovery.

Practical answer

Duplicate `event_id` should not create a duplicate business effect. The malformed line must become visible quarantine evidence with source coordinates/reason. The late event must follow an explicit event-time/business correction policy rather than being silently discarded.

Reconciliation: total observed input = accepted valid records + quarantined records for the bounded evidence window; unique event/business keys satisfy the target invariant; late/corrected events have an auditable treatment. Keep the raw input unchanged so the exercise remains replayable.

G05 REVIEW A - IMPLEMENTATION RUBRIC

Protected runtime rubric A

PASS implementation requires: immutable input; malformed line visibly quarantined; E2 dedupe/idempotency proven; E3 late policy stated and evidenced; deliberate sink failure stays uncommitted/recoverable; recovery rerun succeeds; changed-input retry succeeds; evidence JSON and logs agree.

Conceptual pass requires correct layer boundaries, key/order reasoning, consumer side-effect idempotency, replay plan, Delta repair reasoning, checkpoint ownership and lag diagnosis. Record implementation and reasoning scores separately.